Faculty of Sciences

Department of Computer Sciences

Preparatory work for the master thesis

Procedural Generation of Solvable Levels in Multi-Agent Reinforcement Learning Environment Using Curriculum Learning

Author:
Charels Hugo
hugo.charels@ulb.be

Supervisors:
Lenaerts Tom
Molinghen Yannick

Academic year 2024-2025

Contents

1 Introduction ... 1
1.1 Context ... 1
1.1.1 Laser Learning Environment ... 1
1.2 Motivation ... 2
1.3 Problem statement ... 3
1.4 Structure Overview ... 4
1.5 Abreviation ... 5
2 Problem Formulation ... 6
2.1 Scientific Questions ... 7
2.2 Challenges ... 7
3 State of the Art ... 8
3.1 Procedural Content Generation ... 8
3.1.1 Noise-Based Methods ... 8
3.1.1.1 White Noise ... 8
3.1.1.2 Value Noise ... 9
3.1.1.3 Gradient-Based Noise (Perlin, Simplex, OpenSimplex) ... 10
3.1.1.4 Worley / Voronoi Noise (a.k.a. Cellular Noise) ... 10
3.1.2 Rule-Based Methods ... 11
3.1.2.1 Cellular Automata ... 11
3.1.2.1.1 Parameters and their Effects ... 12
3.1.2.1.2 Use in LLE ... 12
3.1.2.1.3 Example Levels Generated with Cellular Automata ... 12
3.1.2.2 Prefabs ... 13
3.1.2.3 Wave Function Collapse ... 13
3.1.2.3.1 Applicability to LLE ... 13
3.1.3 Learning-Based Methods ... 14
3.1.3.1 Generative Adversarial Networks ... 14
3.1.3.1.1 Applicability to LLE ... 15
3.1.3.2 Variational Autoencoders ... 15
3.1.3.2.1 Sequential Segment-Based Generation ... 16
3.1.3.2.2 Applicability to LLE ... 16
3.1.3.3 Reinforcement Learning-Based PCG ... 16
3.1.3.3.1 Approaches and Architectures ... 17
3.1.3.3.2 Applicability to LLE ... 17
3.2 Solvability ... 18
3.2.1 Gadget-Based Framework for NP-Hardness ... 18
3.2.2 Applying the Method to LLE ... 20
3.3 Curriculum Learning ... 20
3.3.1 Relevance of CL to PCG via RL ... 20
3.3.2 Example Applications of CL in PCGRL ... 21
3.3.3 Future Directions ... 21
4 Methodology ... 23
4.1 Overview of Approach ... 23
4.2 Phase 1: Complexity Proof and Theoretical Foundation ... 23
4.3 Phase 2: Method Development and Selection ... 24
4.3.1 PCG Technique Evaluation ... 24
4.3.2 Solvability Verification System ... 24

4.3.3 Integration Architecture 24
4.4 Implementation details & tools 24
4.5 Risk analysis & mitigation 24
4.6 Timeline 25
5 Conclusion 27
5.1 Summary of Proposed Research 27
5.2 Expected Implications for Multi-Agent Learning 28
5.3 Anticipated Challenges and Next Steps 28
5.4 Concluding Remarks 29
Bibliography 30

Chapter 1

Introduction

1.1 Context

Procedural Content Generation (PCG) refers to the algorithmic creation of data, environments, or assets, often with limited or no direct human input. Initially popularized in the domain of video games to generate maps, levels, textures, and narratives dynamically, PCG has evolved into a powerful paradigm for automating and diversifying content creation across multiple domains. Its advantages include scalability, variability, adaptability, and reduced reliance on manual design. Beyond games, PCG techniques are now employed in simulation-based training environments, robotics, virtual reality, and design automation, where they can rapidly generate diverse scenarios for testing or training purposes.

In parallel, Reinforcement Learning (RL) has emerged as a robust framework for training agents to make decisions through interaction with an environment. Extending this to Multi-Agent Reinforcement Learning (MARL) involves multiple agents learning simultaneously within a shared environment. These agents may cooperate, compete, or coexist independently, and their actions can influence each other's learning dynamics. MARL has found applications in complex settings such as autonomous driving, robotic swarm coordination, strategic games, and resource management problems.

Curriculum Learning is a training strategy inspired by the way humans learn: starting with simpler concepts and gradually progressing to more complex ones. In machine learning, and particularly in reinforcement learning contexts, curriculum learning involves structuring the training process by initially exposing the agent to easier tasks or environments and incrementally increasing the difficulty. This approach aims to accelerate learning, improve convergence stability, and lead to more generalizable behaviors. Curriculum learning is especially valuable when dealing with sparse rewards, complex action spaces, or environments where exploration is otherwise inefficient.

Together, these concepts form a foundation for scalable and adaptive agent training systems. By leveraging PCG to generate environments of varying complexity and integrating curriculum learning to guide the training trajectory, researchers aim to develop more robust and capable agents in multi-agent settings.

1.1.1 Laser Learning Environment

The LLE [1] and its implementation [2], is a 2D grid-based cooperative puzzle game developed as a benchmark for MARL. It supports 1 to 4 agents that must navigate levels filled with static walls and colored laser beams. A central mechanic of LLE is color-matching coordination: each laser beam can only be blocked by an agent of the same color. This rule introduces a high degree of interdependence between agents, requiring them to assist one another in navigating the environment.

Each level is considered complete only when all agents reach their respective exit tiles simultaneously, enforcing strict temporal synchronization. To succeed, agents must execute carefully ordered, cooperative actions—such as positioning to block lasers for teammates or waiting until another agent completes a prerequisite move. These gameplay mechanics define a space where perfect coordination is essential and cannot be bypassed through individual effort alone.

A distinctive property of LLE is its zero-incentive dynamics: intermediate cooperative steps required to progress yield no direct reward. This means agents must learn to perform complex joint behaviors without receiving incremental feedback, making exploration especially difficult. These dynamics give rise to state space bottlenecks—crucial points in a level that require coordinated agent behavior to traverse but offer no local reward signals to guide learning.

LLE has been used to evaluate several state-of-the-art value-based MARL algorithms, including methods augmented with prioritized experience replay, n-step returns, and intrinsic curiosity via random network distillation. Despite these enhancements, agents frequently struggle to escape bottlenecks or explore adequately in LLE, even when they achieve near-perfect coordination in more conventional cooperative settings. These findings underscore LLE's relevance as a testbed for evaluating coordination under sparse and delayed reward signals.

By design, LLE offers a structured yet flexible environment for studying cooperative behavior, with challenges emerging from both spatial constraints and temporal dependencies among agents. Its mechanics support a wide range of level configurations, making it a suitable foundation for procedural level generation targeting collaborative MARL.

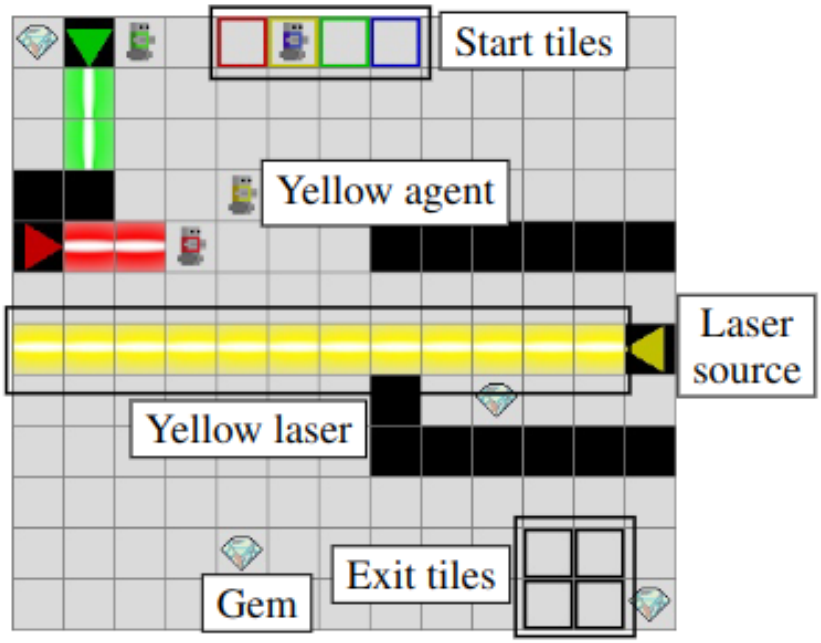


Figure 1: Annotated example of a LLE level

1.2 Motivation

In MARL, environments that require coordination, interdependence, and exploration play a crucial role in evaluating and developing intelligent agents. However, the process of manually designing such environments—especially ones that are both challenging and solvable—is time-consuming and inherently limited in diversity. This is where procedural content generation (PCG) becomes a powerful tool: it enables the automatic creation of a wide variety of levels with controllable properties, fostering generalization and robust policy learning.

Ensuring that generated levels are solvable is essential. In cooperative MARL tasks, unsolvable or poorly designed levels may lead to misleading training signals or dead-end exploration, stalling learning altogether. Unlike single-agent settings, multi-agent environments often involve tight coordination constraints, where one agent's progress depends on the timely actions of others. Solvability, in this context, does not simply mean the presence of a valid path, but the existence of a feasible sequence of synchronized and interdependent actions that lead all agents to jointly complete the task. Automatic solvability checking or solvability-aware generation is therefore a key requirement for meaningful training and evaluation.

While human-designed levels can be carefully crafted to illustrate specific behaviors or challenges, they tend to lack scalability and diversity. Human intuition may also fail to capture edge cases or systematically vary structural properties of coordination. Moreover, relying exclusively on handcrafted levels may lead to overfitting and limited generalization of MARL agents to unseen scenarios. By contrast, PCG methods allow for dynamic curriculum learning, adaptation to agent capabilities, and controlled difficulty scaling—all critical for learning in sparse-reward and high-interdependence settings.

The Laser Learning Environment stands out as a particularly suitable benchmark for this line of work. Its unique mechanics—such as color-matched laser blocking, simultaneous exits, and zero-incentive bottlenecks—place strong emphasis on perfect coordination without intermediate rewards. This makes it not only a difficult challenge for current MARL algorithms, but also a fertile ground for generating levels that target specific coordination patterns, exploration problems, or agent dependencies. LLE's well-defined structure and grid-based representation make it amenable to PCG techniques, while its difficulty profile ensures that generated levels are non-trivial, even for advanced learning agents.

To generate useful and solvable levels for MARL agents, it is also important to consider the learning process of the level generator itself. Directly training a generator to produce highly challenging and coordinated levels from scratch often results in either trivial or unsolvable configurations. This is especially problematic in environments like LLE, where coordination requirements and zero-incentive bottlenecks make solvability highly non-trivial. To address this, curriculum learning can be applied to the level generator, enabling it to progressively learn to design levels of increasing difficulty. Starting from simpler instances with minimal interdependence or reduced agent count, the generator can gradually incorporate more complex coordination constraints, guiding both its own development and the training of MARL agents. This staged approach mirrors how humans teach—building understanding step-by-step—and can significantly improve the efficiency, stability, and diversity of both the generated content and the learning of the agents that interact with it.

1.3 Problem statement

The central problem addressed in this work is the development of a theoretical and conceptual framework for PCG in cooperative puzzle games, with a focus on environments used in MARL. The core challenge is to formalize and implement generation strategies that can produce levels with meaningful cooperative structure—without relying on brute-force methods or manual curation. This work specifically aims to address the following questions:

- How can we generate levels that are provably solvable, without resorting to exhaustive enumeration or post hoc solvability testing? This involves designing generation procedures that embed solvability into the construction process itself.
- How can we assess and promote learnability, ensuring that the generated levels are not only solvable in principle but also amenable to learning by reinforcement learning agents? This requires balancing complexity with clarity, and potentially adapting level difficulty to the agent's current capabilities.
- How can we enforce cooperative constraints, so that solving the level genuinely requires inter-agent coordination, rather than allowing solutions through independent or trivial strategies? The goal is to embed structural dependencies that necessitate synchronized or interdependent actions between agents.

By tackling these questions, this work seeks to contribute toward the automatic generation of structured, meaningful, and challenging multi-agent environments—especially in the Laser Learning Environment, where coordination is both essential and difficult to learn.

1.4 Structure Overview

This document is structured as follows:

- **Chapter 1:** introduces the context and motivation of PCG withing MARL evironment.
- **Chapter 2:** presents the problem formulation and details the scientific questions this thesis aims to explore.
- **Chapter 3:** provides a review of the state of the art, covering existing work on procedural content generation, solvability of 2D games and curriculum learning.
- **Chapter 4:** present the methodology to follow to accomplish the goal of the thesis.

The document also includes a conclusion and a bibliography of all cited works.

1.5 Abreviation

LLE	Laser Learning Environment
PCG	Procedural Content Generation
RL	Reinforcement Learning
CL	Curriculum Learning
MARL	Multi Agent Reinforcement Learning
PCGML	Procedural Content Generation via Machine Learning
PCGRL	Procedural Content Generation via Reinforcement Learning
GAN	Generative adversarial network
CESGAN	Conditional Embedding Self-Attention Generative Adversarial Network
VAE	Variational Autoencoders
WFC	Wave Function Collapse
MDP	Markov Decision Process
CA	Cellular Automata
AI	Artificial Intelligence

Chapter 2

Problem Formulation

Procedural generation of puzzle levels is a well-studied problem, but ensuring that these levels are both solvable and meaningful—particularly in multi-agent cooperative settings—remains a significant challenge. In environments like the Laser Learning Environment, this difficulty is amplified due to the need for precise coordination between agents.

A level may be technically solvable if each agent can independently reach its goal, but such a design fails to leverage the cooperative dynamics that define the environment. On the other hand, levels that demand coordination but are inherently unsolvable—due to flawed structure, deadlocks, or unintended constraints—are unusable for both human players and learning agents. Striking a balance between solvability and cooperation is non-trivial and lies at the heart of the level design problem in MARL.

This thesis aims to formalize and address this challenge by laying the groundwork for a level generation framework that satisfies three essential and interrelated criteria:

1. **Solvability** – The level must admit at least one valid sequence of actions through which all agents can reach their respective goals simultaneously. This guarantees the level can be completed in principle.
2. **Learnability** – The solution should be discoverable through exploration, gameplay, or reinforcement learning. Levels should provide structure or affordances that guide agents (or players) toward coordinated solutions without requiring exhaustive search.
3. **Cooperation** – The level must inherently require inter-agent coordination. That is, solving the level must necessitate the use of cooperative mechanics (e.g., laser blocking in LLE), ruling out trivial solutions that involve only individual action.

These principles are illustrated below, using examples from LLE level generation:

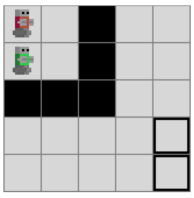


Figure 2: Unsolvable: No path.

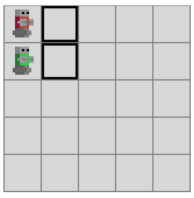


Figure 3: Uninteresting: Solvable without cooperation.

Figure 4: Desirable: Solvable and requires coordinated agent behavior.

2.1 Scientific Questions

This work investigates several core scientific questions aimed at grounding and guiding the design of cooperative level generators:

- How can we procedurally generate levels that are both diverse and provably solvable in multi-agent environments?
- What metrics can be developed to quantify solvability, cooperation requirements, and difficulty?
- How does level structure influence the learnability of tasks by MARL agents?
- Can curriculum learning be integrated into the level generation process to progressively increase challenge while maintaining solvability?
- What are the trade-offs between randomness in generation and the generalization capabilities of agents trained on those levels?

By addressing these questions, the thesis contributes to a better understanding of how PCG can be used not only to generate content, but to scaffold learning, evaluate agent capabilities, and enable better generalization across tasks.

2.2 Challenges

Designing a procedural generation framework for cooperative MARL environments introduces a set of non-trivial challenges:

- **Solvability without overfitting** – Ensuring that levels are solvable without embedding specific hardcoded solutions or patterns that agents can exploit, thereby preserving generality.
- **Maintaining structural diversity** – Avoiding repetitive or overly similar level layouts to prevent agent overfitting and support exploration and generalization.
- **Multi-agent coordination complexity** – Capturing the nuanced dynamics of inter-agent cooperation, including synchronization, dependency chains, and role-switching.
- **Difficulty calibration and scaling** – Generating levels with controllable difficulty to support curriculum learning, enabling both agents and generators to progress incrementally.
- **Formal evaluation metrics** – Defining and validating quantitative metrics for evaluating solvability, learnability, cooperation, and diversity of generated content.
- **Scalability and efficiency** – Ensuring that the generation process is computationally tractable, especially in settings where large volumes of levels are needed to train or benchmark learning agents.

Overcoming these challenges is central to the development of a principled, usable, and impactful PCG framework for cooperative MARL environments like LLE.

Chapter 3

State of the Art

3.1 Procedural Content Generation

Procedural Content Generation refers to the algorithmic creation of game content such as levels, maps, textures, objects or more. Initially motivated by the need for replayability and reduced manual labor in game design, PCG now plays a vital role in **game development**.

3.1.1 Noise-Based Methods

Noise functions are a fundamental procedural technique for generating spatially coherent randomness. Originally developed for natural texture synthesis and terrain generation, these methods have proven effective for designing 2D levels such as mazes, dungeons, or cave-like environments. Their key advantage lies in balancing structure and randomness: by varying the parameters of the noise function, one can control the density, scale, and smoothness of the generated features.

In the context of level generation for LLE, noise functions can define open areas, walls, paths, or dynamic zones in a continuous yet controllable manner. They are particularly well-suited for generating levels that exhibit global spatial coherence — a property essential for agent coordination and exploration in MARL scenarios.

Below, we review the main types of noise functions used in PCG, focusing on those relevant for 2D grid environments similar to LLE.

3.1.1.1 White Noise

White noise[1] is the simplest form of noise, where each grid cell is assigned an independent random value, typically sampled uniformly. This results in high-frequency, uncorrelated values that resemble static. While it's computationally trivial, the complete lack of spatial correlation makes it unsuitable on its own for meaningful level generation. However, it serves as a crucial primitive for more structured noise functions and post-processing techniques.

[1] https://gameidea.org/2023/12/16/noise-functions/

- **density** (int): The probability (in %) that a given cell becomes a wall. Higher density results in more walls and less navigable space.

In LLE, using white noise directly would produce chaotic and unplayable levels. Nevertheless, it is often used as a base for convolutional smoothing, seed placement in cellular automata, or as a source of entropy for stochastic components in hybrid systems.

Example levels generated with white noise:

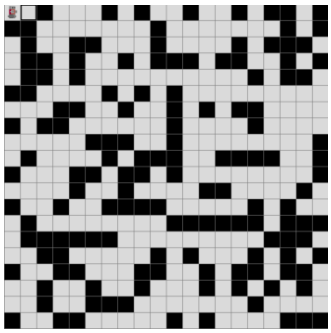


Figure 5: density = 40

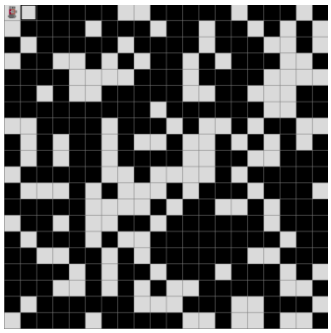


Figure 6: density = 60

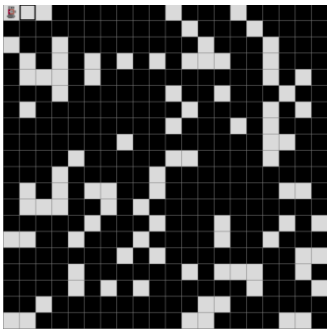


Figure 7: density = 80

3.1.1.2 Value Noise

Value noise[2] improves on white noise by assigning random scalar values to the vertices of a regular grid and interpolating the values between them. In 2D, this interpolation is typically bilinear (or bicubic for smoother results). The result is a continuous noise field with controllable spatial coherence.

- **scale** (float): Controls the "zoom" of the noise — lower values create more variation; higher values create large smooth zones.
- **threshold** (float): Determines the cutoff value for deciding whether a tile is a wall or walkable. Useful for adjusting the openness of the layout.

In LLE, thresholding this smooth field (e.g., value > 0.5 ⟶ wall) yields cave-like open spaces with soft edges and variable density. By layering multiple frequencies of value noise (a technique known as fractal Brownian motion), it is possible to introduce both large-scale structure and fine-grained detail, enabling the generation of complex, navigable levels suitable for training cooperative agents.

Example levels generated with value noise:

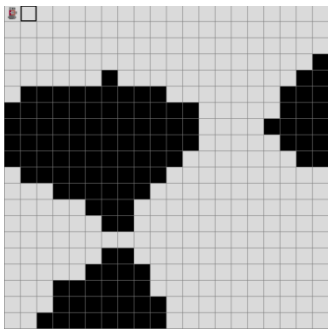


Figure 8:
scale=10.0, threshold=0.0

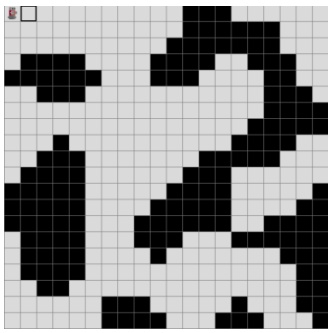


Figure 9:
scale=5.0, threshold=−0.1

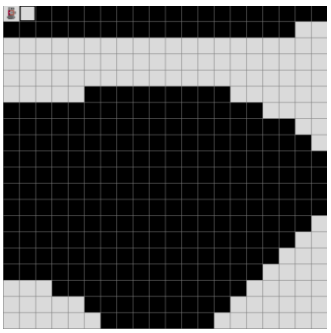


Figure 10:
scale=15.0, threshold=0.1

[2] https://gameidea.org/short-posts/value-noise/

3.1.1.3 Gradient-Based Noise (Perlin, Simplex, OpenSimplex)

Gradient noise methods take value noise a step further by associating each lattice vertex with a pseudo-random gradient vector. A point's noise value is determined by computing the dot product between its relative position and the surrounding gradients, followed by a smooth interpolation.

- **Perlin Noise**[3]: A seminal method for generating coherent noise. It produces smooth gradients and natural-looking variations but is susceptible to directional artifacts due to its underlying grid structure.
- **Simplex Noise**[4]: Designed to address the artifacts of Perlin noise, Simplex noise uses a simplex tessellation (triangular in 2D) rather than a square grid, leading to more isotropic patterns and improved performance, especially in higher dimensions.
- **OpenSimplex Noise**[5]: A patent-free and visually cleaner alternative to Simplex noise, designed to avoid visual artifacts and directional bias in 2D and 3D environments.

Main parameters (same as value noise):
- **scale** (float): Controls spatial frequency of changes.
- **threshold** (float): Defines the walkable/wall boundary.

In LLE, gradient-based noise can be used to define large organic spaces or connectivity patterns between regions. For example, tunnels or open corridors can emerge naturally when applying appropriate thresholds to the noise field. These techniques are also useful in curriculum learning settings, where the degree of noise smoothness can modulate the difficulty of the level layout.

Example levels generated with Perlin noise:

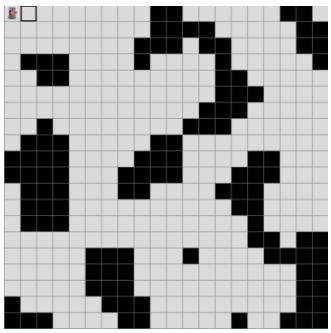


Figure 11:
scale=4.0, threshold=−0.2

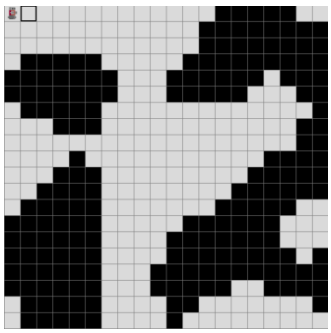


Figure 12:
scale=6.0, threshold=0.0

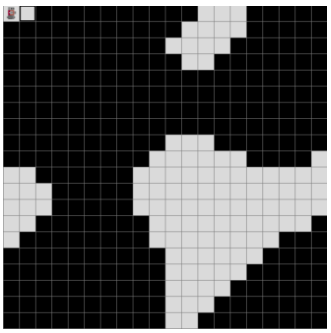


Figure 13:
scale=8.0, threshold=0.2

3.1.1.4 Worley / Voronoi Noise (a.k.a. Cellular Noise)

Worley noise[6] is based on computing distances between each point in the grid and a set of randomly placed seed points. The value of a cell typically reflects its distance to the nearest (or k-nearest) seed, resulting in a cellular structure similar to a Voronoi diagram.

- **num_points** (int): Number of random seed points placed on the grid.

[3] https://en.wikipedia.org/wiki/Perlin_noise
[4] https://en.wikipedia.org/wiki/Simplex_noise
[5] https://en.wikipedia.org/wiki/OpenSimplex_noise
[6] https://en.wikipedia.org/wiki/Worley_noise

- **cutoff** (float): A normalized distance threshold used to decide if a point is close enough to a seed to be walkable. Values are typically between 0.2 and 0.5.

This type of noise is especially well-suited for generating room-and-corridor style levels. Seed points can be interpreted as room centers, while paths connecting them can be extracted using graph algorithms (e.g., Delaunay triangulation followed by A*). The resulting levels are modular and interpretable, properties that are desirable in MARL environments such as LLE where coordinated navigation is crucial.

Example levels generated with Worley noise:

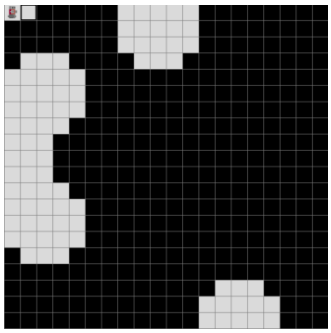


Figure 14: points=5, cutoff=0.1

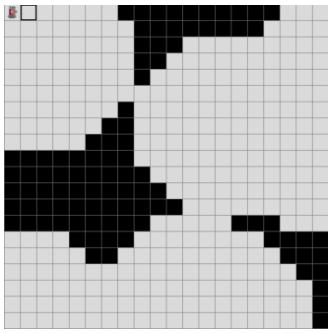


Figure 15: points=8, cutoff=0.2

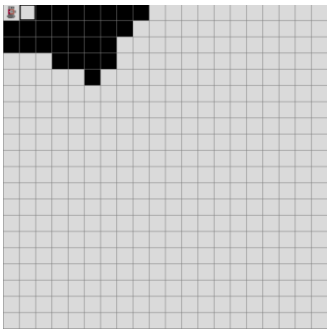


Figure 16: points=12, cutoff=0.3

Noise Type	Spatial Coherence	Level Structure	Use Case in LLE
White Noise	None	Chaotic / Unusable alone	Input to other methods
Value Noise	Low	Smooth blobs/caves	Cave levels, soft rooms
Perlin/Simplex	Medium to High	Flowing terrain, patterns	Organic layouts, open corridors
Worley/Voronoi	High	Cellular / rooms networks	Room generation, modular connectivity

3.1.2 Rule-Based Methods

3.1.2.1 Cellular Automata

CA[7] [3] are rule-based systems in which each cell on a grid evolves over discrete time steps based on the states of its neighboring cells. This local-update mechanism enables the emergence of complex global patterns from simple rules. In PCG, CAs are particularly valued for generating organic, natural, and self-smoothing structures — such as cave systems, erosion effects, or terrain deformations.

A widely used form of CA for dungeon or cave generation is the "cellular cave generator", inspired by Conway's Game of Life [4]. The algorithm typically proceeds in two phases:

[7] https://en.wikipedia.org/wiki/Cellular_automaton

1. **Initialization**: Start with a noisy binary map, such as a grid where each cell has a fixed probability of being a wall.
2. **Iterations**: At each step, for each cell:
   - Count the number of wall neighbors (usually in the 8 surrounding directions).
   - Apply a rule such as:

   "If a cell has ≥ 5 wall neighbors ⟶ it becomes a wall; else ⟶ it becomes empty."

After several iterations, this process transforms random blobs into connected, navigable, cave-like spaces.

3.1.2.1.1 Parameters and their Effects

The behavior and output of a cellular automaton are controlled by a few critical parameters:

- **Initial density** (int):
  - The probability (in %) that a cell is a wall at the start.
  - Higher density ⟶ more initial clutter; lower ⟶ more openness.
- **Number of iterations** (int):
  - Determines how many times the rules are applied.
  - More iterations lead to smoother, more structured levels, but may eliminate small features.
- **Survival rules** (fixed):
  - In most simple generators, the rule is: "A cell becomes wall if ≥ 5 neighbors are wall."
  - This rule can be modified, though the standard 5-wall rule is a good default for 2D caves.

3.1.2.1.2 Use in LLE

In LLE, cellular automata are useful for generating natural cave systems, or exploration-heavy environments where agents must adapt to irregular topologies. Their ability to evolve noisy grids into structured yet unpredictable spaces makes them ideal for training and testing agent coordination under partial observability.

Additionally, post-processing (e.g., flood fill, region pruning, path guarantees) is often required to ensure multi-agent reachability and playability across the entire level.

3.1.2.1.3 Example Levels Generated with Cellular Automata

Using same initial white noises grid with density of 60 but with different iterations, we generated multiple cave-style levels. The results below show how level structure changes as the number of iteration grow.

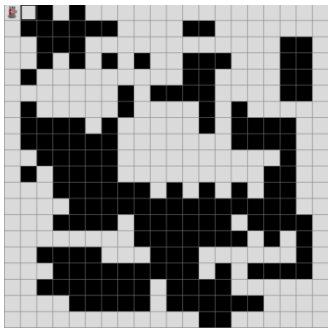


Figure 17: iterations=1

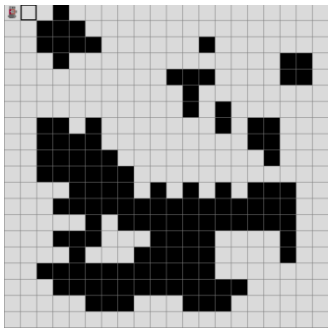


Figure 18: iterations=2

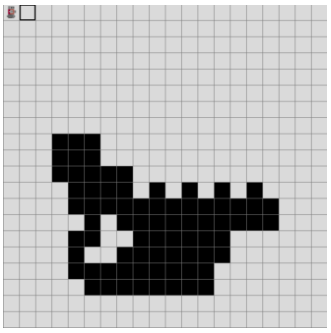


Figure 19: iterations=4

3.1.2.2 Prefabs

Prefab-based generation uses hand-designed or procedurally crafted level fragments (prefabricated structures) that are assembled together to form complete levels. These fragments can range from small 3x3 tiles to larger rooms or hallways, and can be stitched together via predefined connection rules or procedural placement.

This approach gives designers strong control over local structure and gameplay affordances, while still allowing for variation through random selection and positioning of prefabs. It is especially useful when level semantics or agent affordances need to be preserved (e.g., guaranteed corridors, cover zones, chokepoints).

In LLE, prefabs are ideal for generating modular rooms, tactical layouts, and agent-starting zones while ensuring levels remain fair and readable for MARL tasks.

3.1.2.3 Wave Function Collapse

WFC is a constraint-solving algorithm used in procedural content generation that produces novel and coherent patterns by iteratively selecting tile values under local adjacency constraints. Originally introduced by Gumin [5] and further developed by Heese et al. [6], WFC has been widely adopted in game development due to its ability to replicate the stylistic features of a given input while enabling significant variation. It operates through a greedy, non-backtracking process that reduces the entropy (i.e., uncertainty) of each grid cell by propagating adjacency constraints until the grid collapses into a complete and valid configuration.

There are two main WFC variants: the simple tiled model, where adjacency rules are hand-designed, and the overlapping model, which extracts patterns automatically from example inputs. Both versions operate by treating the grid as a set of segments, each with a possible set of values, and iteratively collapsing them into specific tiles while respecting directional constraints based on neighboring tiles.

3.1.2.3.1 Applicability to LLE

WFC is well-suited for generating spatially coherent, tile-based layouts, making it a viable candidate for LLE level generation in the following ways:

- **Grid compatibility:** LLE levels are defined on a 2D grid, matching the structure WFC was designed for.

- **Local constraint modeling:** The adjacency-based logic of WFC can encode constraints such as "a laser can only appear next to a clear path" or "a colored wall must be reachable by an agent of the corresponding color."
- **Pattern diversity:** By training on existing LLE levels, the overlapping WFC model could learn plausible and varied configurations of walls, paths, and lasers.

However, several challenges must be addressed for effective integration:

- **Global coordination constraints**: LLE levels require global dependencies between agents (e.g., one agent must unlock a path for another). WFC's local propagation approach does not naturally capture such long-range interdependencies.
- **Functional tile semantics**: Tiles in LLE carry gameplay functions (blocking lasers, allowing passage), not just visual patterns. This necessitates augmenting WFC with semantic constraints beyond visual adjacency.
- **Playability validation**: WFC cannot guarantee that the generated levels are solvable or require meaningful cooperation without external validation, such as reinforcement learning agents or symbolic solvers.

Potential Enhancements
To better support the requirements of LLE, WFC can be extended or hybridized:
- **Rule-based constraints** could be manually added to encode gameplay logic (e.g., lasers must not create deadlocks).
- **Post-generation validation** using MARL agents can filter out unplayable or trivial levels.
- **Hierarchical or semantic WFC** variants may help capture higher-order dependencies relevant to cooperative puzzles.

In summary, while classical WFC is not directly aware of agent behavior or cooperation, its core principles of constrained tile generation and flexible pattern recombination make it a promising component for LLE level generation—especially when augmented with domain-specific constraints and validation mechanisms.

3.1.3 Learning-Based Methods

Machine Learning approaches to PCG aim to learn level generation patterns directly from data. These methods are especially powerful when many example levels are available or when there is a need to generate levels that adapt to player behavior or learning agents. Here are the main ML techniques used for 2D grid environments:

3.1.3.1 Generative Adversarial Networks

GANs have emerged as a powerful approach for generative tasks in machine learning , particularly in image synthesis and PCG. A GAN consists of two neural networks trained in opposition: a generator, which learns to produce plausible outputs from noise, and a discriminator, which learns to distinguish between real and generated data. Through adversarial training, the generator improves its output quality until the discriminator can no longer reliably distinguish between real and generated samples.

While GANs have shown strong results in generating visually convincing outputs, their application to game level generation presents unique challenges. Unlike image data, game levels often require structural coherence and functional correctness — a level must not only look plausible but also be

playable, adhering to game-specific rules and mechanics. This functional requirement significantly increases the complexity of level generation tasks.

To address this, recent work has extended traditional GANs into more sophisticated architectures [7]. One such model is the Conditional Embedding Self-Attention GAN (CESAGAN), which integrates:

- **Self-attention mechanisms** to model long-range spatial dependencies (important for ensuring distant elements in a level work together, like keys and doors),
- **Conditional embeddings** that encode desired level properties (e.g. number of enemies, walls, or specific gameplay constraints), and
- A **bootstrapping mechanism** where only playable generated levels are added back into the training data, expanding the dataset and improving playability over time.

These techniques allow GANs to generate levels that are not just visually similar to human-designed ones but are also functionally sound.

3.1.3.1.1 Applicability to LLE

Applying GANs like CESAGAN to LLE is plausible but would require several adaptations:

1. **Functional Constraints Encoding**: LLE levels involve constraints like color-matching and coordinated agent positioning. These can be encoded as part of a conditional vector (analogous to the CESAGAN's vector u) representing key properties of a level (e.g., number of lasers per color, agent positions, beam crossings). This would enable conditioning the generator to produce functionally coherent levels.
2. **Non-Local Dependencies**: Since LLE emphasizes coordination across space (e.g., agent A must block a beam so agent B can pass), self-attention mechanisms would help capture such long-range interdependencies between level elements.
3. **Playability Verification**: CESAGAN's bootstrapping technique—filtering generated levels based on playability—can be adapted to LLE using either hand-coded heuristics (e.g., reachability checks) or automated agents that simulate multi-agent cooperation to verify solvability.
4. **Data Scarcity**: If LLE lacks a large corpus of human-designed levels, bootstrapping is particularly valuable. CESAGAN demonstrated that starting with as few as five human-designed levels, it could generate diverse, playable content via iterative self-training.

3.1.3.2 Variational Autoencoders

VAEs are generative models that learn to encode high-dimensional data into a continuous latent space and decode it back into meaningful outputs. Recent advances in this area, such as the work by Sarkar et al. [8], have shown particular promise for sequential level generation. Introduced by Kingma and Welling, VAEs consist of two neural networks: an encoder that maps data to a latent representation, and a decoder that reconstructs data from this latent space. Training is achieved by optimizing a loss function composed of a reconstruction error and a Kullback-Leibler (KL) divergence term, the latter ensuring that the latent space approximates a known prior (typically a Gaussian distribution).

VAEs have been widely adopted in procedural content generation via machine learning (PCGML) for

games, especially for generating 2D platformer levels such as in Super Mario Bros.[8], Mega Man[9], and Kid Icarus[10]. One key advantage of VAEs is the continuity of their latent space, which enables smooth interpolation, blending, and controlled sampling—crucial for generating diverse and coherent game content.

3.1.3.2.1 Sequential Segment-Based Generation

Traditional VAE-based approaches often operate on fixed-size level segments generated independently. However, this method can lead to incoherent levels when segments are naively stitched together. The article introduces a novel sequential generation method, where the VAE is trained not to reconstruct the input segment, but to generate the next segment in the level's progression. This enables the generation of logically coherent sequences of level segments. Additionally, a classifier predicts the spatial placement (up, down, left, right) of each segment, allowing for level structures that extend in multiple directions.

This architecture proved effective for multi-directional games like Mega Man and blended environments combining different games. It also demonstrated the potential for creating arbitrarily long and coherent levels—a key requirement for robust PCG systems.

3.1.3.2.2 Applicability to LLE

VAEs—particularly in their sequential segment-based form—could serve as a strong foundation for LLE level generation:

- Multi-directional progression: Like Mega Man levels, LLE maps require flexible structural layouts (e.g., vertical and horizontal dependencies), which the VAE-classifier pipeline supports.
- Coherence and continuity: Generating playable LLE levels requires maintaining logical progression across segments (e.g., correctly positioned lasers, walls, and coordination points), a need directly addressed by sequential VAEs.
- Blending and diversity: VAEs enable interpolation and controlled sampling in latent space, which could be used to generate levels of varying difficulty, symmetry, or coordination requirements.
- Scalability: Arbitrarily long levels can be generated by iterative decoding, allowing the exploration of larger environments suitable for complex MARL tasks.

However, adapting VAEs to LLE presents unique challenges. Unlike platformers where level validity is mostly visual and path-based, LLE level validity depends on agent interaction and cooperative solvability. Therefore, future work may require conditioning VAEs not just on structural patterns, but also on gameplay constraints, such as solvability by a team of agents with complementary roles.

3.1.3.3 Reinforcement Learning-Based PCG

RL has emerged as a promising technique for PCG, as comprehensively surveyed by Khalifa et al. [9], enabling the automatic creation of game content by training agents that learn to generate environments through interaction with a reward-driven system. Unlike generative models such as

[8] https://en.wikipedia.org/wiki/Super_Mario_Bros.
[9] https://en.wikipedia.org/wiki/Mega_Man
[10] https://en.wikipedia.org/wiki/Kid_Icarus_(series)

VAEs or GANs, which learn to recreate or interpolate data distributions, RL-based PCG treats content generation as a sequential decision-making process.

In this paradigm, the content generator is framed as an RL agent whose actions correspond to modifications of the content (e.g., placing tiles, adding game objects), and whose reward signal is designed to guide desirable properties, such as solvability, difficulty, diversity, or player engagement. This approach is especially suitable for generating structured, constrained environments where satisfying high-level objectives is crucial.

3.1.3.3.1 Approaches and Architectures

The survey identifies several RL-based PCG strategies:

1. Simulation-Based Reward Functions
   Content is evaluated through simulations (e.g., A* pathfinding, gameplay heuristics) to determine playability and challenge. These evaluations form the reward signal guiding the RL agent.
2. Agent-Environment Framework
   The generator and player can be modeled as separate agents interacting with the same environment, enabling co-adaptation. This is particularly relevant for multi-agent settings, such as cooperative games.
3. Curriculum and Adaptive Content
   RL allows the dynamic generation of content tailored to the player's or agent's skill level, supporting difficulty adjustment and curriculum learning.
4. Level Representation as Markov Decision Processes (MDPs)
   Content generation is formalized as an MDP, where states represent partial content (e.g., partial levels), actions modify that content, and transitions reflect the construction process.

The use of RL also enables interactive and adaptive PCG: levels can be generated in response to real-time performance or learning trajectories, making RL a powerful tool for creating tailored or evolving environments.

3.1.3.3.2 Applicability to LLE

- Multi-Agent Coordination as a Design Goal
  RL agents can be trained to construct levels that require specific types or degrees of coordination, by rewarding solutions that depend on multiple agents working together to solve puzzles.
- Solvability and Complexity via Simulation
  Simulated agents (either scripted or learned policies) can be used to test level solvability during generation. The RL reward can be shaped to favor levels solvable only with collaboration, ensuring alignment with LLE's core mechanics.
- Curriculum and Progressive Difficulty
  RL-based generators can incrementally increase complexity, supporting curriculum learning in MARL contexts. This could produce levels that gradually require tighter coordination, encouraging skill acquisition over time.

- Dynamic Content for Training Agents
  Since LLE is a benchmark for MARL, an RL-based generator could continuously supply novel levels, enabling agents to avoid overfitting and generalize better to unseen tasks.
- Interactive Design Tools
  RL agents could assist human designers by proposing partial levels or completing unfinished ones, especially when optimizing for coordination complexity or specific constraints.

In summary, RL-based PCG offers a powerful and flexible approach for generating levels in LLE, particularly when the goal is to enforce or explore inter-agent dependencies. Unlike VAEs that prioritize structural coherence, RL-PCG allows designers to specify and reward functional constraints, making it ideal for MARL benchmarks like LLE where coordination is not just desired, but required.

3.2 Solvability

In single-agent games, verifying the solvability of a level often reduces to finding a path from a starting point to a goal, akin to solving a maze. However, when additional mechanics such as complex object interactions or multi-agent dynamics are introduced, solvability checking becomes substantially harder. In such cases, even deciding whether a level is solvable becomes a computationally challenging decision problem.

A prominent example of this phenomenon is demonstrated by Aloupis et al. [10], who show that a large class of well-known video games are NP-hard in their generalized form. This includes games such as Super Mario Bros., Donkey Kong Country[11], The Legend of Zelda[12], Metroid[13], and Pokémon[14]. In each case, they consider a generalization of the original game where the size of the level can grow arbitrarily large, but the rules and core mechanics remain unchanged. The authors define the following decision problem:

Reachability Problem: Given a level (represented as a game state), is it possible to reach the designated goal location from the designated start location?

This problem is shown to be NP-hard by a reduction from the classical NP-complete problem 3-SAT, using a modular construction that encodes the satisfiability of a Boolean formula into a game level.

3.2.1 Gadget-Based Framework for NP-Hardness

The reduction method relies on creating logical gadgets within the game world that simulate the behavior of Boolean logic. These gadgets are small, localized regions of the level that implement specific logical operations or constraints using only the legal game mechanics. The general structure of the reduction is as follows:

1. High-Level Structure:
   The reduction encodes an instance of 3-SAT as a level where the player must:
   - Make exclusive binary choices corresponding to truth assignments for variables.
   - Activate paths corresponding to literals (either positive or negative) of those variables.

[11] https://en.wikipedia.org/wiki/Donkey_Kong_Country
[12] https://en.wikipedia.org/wiki/The_Legend_of_Zelda
[13] https://en.wikipedia.org/wiki/Metroid
[14] https://en.wikipedia.org/wiki/Pok%C3%A9mon

- Ensure that each clause in the formula is "satisfied" (i.e., made traversable).
- Finally, traverse a "check path" that visits all clauses and leads to the goal, but only if all clauses are unlocked.

This structure is shown schematically in the paper's Figure 1 (not reproduced here), which illustrates a linear progression through variable choices, followed by clause satisfaction, and finally goal validation.

2. Gadget Types and Their Purpose:
   Each component of the 3-SAT formula is implemented using a corresponding gadget:

   - **Variable Gadget**
     Each variable in the formula is associated with a gadget that forces the player to commit to one of two mutually exclusive paths — either assigning the variable to true or to false. Once a choice is made, the alternative path becomes permanently inaccessible (e.g., by using one-way drops or destructible paths). These two paths then lead to the clause gadgets corresponding to the literal (x) and its negation $(\neg x)$.
   - **Clause Gadget**
     Each clause is implemented as a gadget that initially blocks the player from traversing a "check path" that passes through it. However, the gadget can be unlocked from any of its three incoming literal paths. Visiting the clause from one of its literals (e.g., from the variable gadgets) performs an action (like pressing a switch or releasing a key) that permanently opens the clause's check path. This simulates satisfying a clause by at least one literal.
   - **Check Path**
     After choosing variable assignments and visiting the relevant clauses, the player must traverse a final corridor that passes through all clause gadgets. If any clause is not satisfied (i.e., its path is still locked), then the player cannot proceed to the goal. This enforces the global constraint that all clauses must be satisfied to reach the goal.
   - **Crossover Gadget**
     In 2D grid-based levels, wires corresponding to literal paths often need to cross. Crossover gadgets are used to simulate wire crossing without leakage (i.e., without allowing the player to switch between paths). These gadgets are constructed using carefully constrained one-way interactions or game physics to prevent unintended traversal. Importantly, only unidirectional crossings are needed, and even then, only once per path — which simplifies implementation.
   - **Start and Finish Gadgets**
     These typically enforce preconditions on the player's state. For example, in Super Mario Bros., the player is forced to pick up a Super Mushroom at the start so that they are "big Mario," which is required to break bricks in later gadgets.

3. Correctness Argument
   The construction ensures the following equivalence:

   - If the original 3-SAT formula is satisfiable, then there exists a sequence of decisions (variable assignments) that leads to unlocking all clause gadgets. Hence, the check path is traversable and the goal is reachable.
   - If the formula is not satisfiable, then at least one clause remains locked, and the check path cannot be fully traversed. The goal is unreachable.

   Therefore, deciding whether the level is solvable is equivalent to solving the 3-SAT instance, establishing NP-hardness.

3.2.2 Applying the Method to LLE

This method is highly adaptable to environments like LLE, which offer:

- Binary irreversible choices (e.g., agents committing to one route that disables the alternative),
- Conditional traversal (e.g., an agent can only pass if another agent is blocking a laser of matching color),
- Persistent state change (e.g., lasers being disabled by agent placement),
- Geometric layout constraints in a 2D grid,

In the context of LLE, the same gadget construction strategy can be applied:

- **Variable gadgets** can be implemented as choice corridors for individual agents, where the agent must decide which sub-path to enter, thereby encoding a truth value.
- **Clause gadgets** can be implemented as lasers that must be blocked by specific agents to open a corridor for another agent to continue. If no agent visited the clause (i.e., set a literal to true), the check agent cannot proceed.
- **Crossover gadgets** can be implemented using walls and timing constraints to prevent agents from interfering across paths.
- **Multi-agent constraints** in LLE (e.g., one agent waiting for another to hold a beam) make the framework even more expressive, allowing simulation of more complex logic.

This analysis supports the hypothesis that deciding whether a given LLE level is solvable is NP-hard.

3.3 Curriculum Learning

CL is a training strategy inspired by the natural progression of human education: rather than learning a complex task from scratch, an agent is guided through a structured sequence of simpler tasks that gradually increase in difficulty. In RL, this idea has gained traction as a means of accelerating training and improving final performance in environments that would otherwise be too difficult to learn directly.

As discussed in the comprehensive survey by Narvekar et al. [11], a curriculum in RL refers to a sequenced arrangement of tasks or experience samples designed to accelerate learning. These tasks may differ in complexity, reward structure, state/action spaces, or transition dynamics. The key elements of a CL method are:

- **Task Generation**: Creating intermediate tasks that bridge the gap between a randomly initialized policy and success in the target task.
- **Sequencing**: Determining the order in which the tasks or data samples are presented.
- **Transfer Learning**: Leveraging knowledge from earlier tasks to improve performance on subsequent ones.

Curriculum structures can range from simple linear sequences (e.g., Task A $\longrightarrow$ Task B $\longrightarrow$ Target Task) to more complex directed acyclic graphs of tasks. Evaluation metrics for CL often include jumpstart performance, time-to-threshold, and asymptotic performance.

3.3.1 Relevance of CL to PCG via RL

The PCGRL framework developed by Khalifa et al. [9] frames PCG as a MDP, allowing RL agents to iteratively generate game levels by modifying small parts of the environment. While this allows for learning without prior data and supports interactive generation, training such agents from scratch can be very challenging due to sparse or delayed reward signals and the large action/state space inherent to content design.

Curriculum Learning can address these challenges in several ways:

- **Reducing Reward Sparsity**: Intermediate tasks can provide denser rewards by training agents on simpler sub-problems (e.g., placing a single player tile correctly before optimizing enemy layouts).
- **Simplifying Exploration**: By initially focusing on smaller or more constrained versions of the level generation task (e.g., reduced grid size, fewer object types), agents can learn useful priors.
- **Structured Skill Acquisition**: Agents can incrementally learn level design primitives (e.g., connectivity, solvability, aesthetics) before being tasked with full-level generation.
- **Improving Generalization**: Curriculum learning may lead to more robust generators capable of producing diverse and high-quality content across different game genres or styles.

3.3.2 Example Applications of CL in PCGRL

While CL has not yet been explicitly implemented in PCGRL, the framework naturally supports it.

- In the Sokoban[15] environment, a curriculum could begin with levels containing only one box and goal, progressively increasing complexity.
- In the maze generation task, early tasks could reward agents for simply creating connected paths, before introducing goals like maximizing path length or adding constraints (e.g., specific start/end locations).
- In Zelda-style environments, agents might first learn to place doors and keys in valid configurations before introducing enemies or complex spatial layouts.

The modular design of the PCGRL environment—with separate Problem, Representation, and Change Percentage components—also facilitates curriculum design. One could dynamically generate intermediate problems with simpler constraints and smoothly escalate the difficulty via parameters like allowed tile types, number of editable tiles, or level size.

3.3.3 Future Directions

Combining Curriculum Learning with PCGRL opens avenues for:

- Automated curriculum generation, where intermediate tasks are discovered dynamically based on agent progress or learning gradients.
- Transferable level design policies, where curricula train agents to generalize design principles across games or genres.
- Mixed-initiative design, where a human designer co-constructs curricula to guide the agent toward creative or stylistically coherent outputs.

[15] A franchise of puzzle video games in which the player pushes boxes around in a warehouse, trying to get them to storage locations. https://en.wikipedia.org/wiki/Sokoban

In summary, Curriculum Learning offers a powerful and underexplored enhancement for reinforcement learning-based procedural content generation. Integrating it into the PCGRL pipeline could significantly improve both the training process and the quality of the generated content.

Chapter 4

Methodology

This thesis adopts a multi-phase experimental-theoretical approach that systematically addresses the three core challenges identified in the problem formulation: solvability, learnability, and cooperation requirements in procedurally generated multi-agent environments. The methodology is structured into four interconnected phases, each building upon the theoretical and empirical foundations established in the previous stages.

4.1 Overview of Approach

The research follows a sequential methodology designed to establish both theoretical foundations and practical implementations:

Phase 1: Theoretical Foundation
Formal complexity analysis of LLE solvability and establishment of theoretical bounds.

Phase 2: Method Development and Selection
Evaluation of PCG techniques, development of hybrid generation framework, and implementation of solvability verification systems.

Phase 3: Experimental Validation
Empirical testing of generation methods, comparative analysis of training paradigms, and curriculum learning integration.

Phase 4: Analysis and Documentation
Results synthesis, interpretation, and thesis composition.

4.2 Phase 1: Complexity Proof and Theoretical Foundation

The first step establishes the computational complexity of determining whether a given LLE level is solvable. Following the gadget-based reduction framework established by Aloupis et al., We will demonstrate that the LLE solvability decision problem is NP-hard through a polynomial-time reduction from 3-SAT.

The approach involves designing LLE-specific logical gadgets using the environment's native mechanics and constructing a polynomial-time mapping from any 3-SAT formula to an LLE level.

This will provide theoretical justification for the computational challenges inherent in generating solvable levels.

4.3 Phase 2: Method Development and Selection

4.3.1 PCG Technique Evaluation

This phase systematically evaluates multiple procedural content generation approaches to identify the most suitable methods for LLE level generation. The evaluation will cover three main categories:

- **Noise-Based Methods**: White Noise, Value Noise, Perlin/Simplex Noise, and Worley Noise
- **Rule-Based Methods**: Cellular Automata, prefab-based systems, and Wave Function Collapse
- **Learning-Based Methods**: GAN-based generators, VAE approaches, PCGRL

Methods will be assessed based on controllability, compatibility with solvability constraints, computational efficiency, output diversity, and integration potential.

4.3.2 Solvability Verification System

A multi-layered verification system will be developed to ensure generated levels meet solvability requirements:

- **Symbolic Verification**: Graph-based pathfinding, constraint satisfaction solvers, and SAT-based verification
- **Agent-Based Verification**: MARL agents testing level completion with timeout-based classification
- **Hybrid Validation**: Combined approaches with confidence scoring and feedback loops

4.3.3 Integration Architecture

Design a unified system architecture that integrates procedural generation components, solvability verification modules, curriculum learning controllers, and performance evaluation frameworks. This architecture will support both standalone level generation and dynamic curriculum adaptation based on agent learning progress.

4.4 Implementation details & tools

- Languages: Python, Rust
- Compute: local GPU(s) + optional cloud for large training runs. Version control with Git/GitHub.
- Reproducibility: include seed control, publish generator code + datasets, save final model checkpoints.

4.5 Risk analysis & mitigation

Several potential risks may affect the successful completion of this work. First, proving the LLE solvability of generated levels could be significantly more challenging than anticipated. If the previously proposed approach for demonstrating solvability proves ineffective, alternative methods will need to be investigated, potentially requiring substantial additional theoretical or empirical work.

Second, the procedural content generation (PCG) techniques presented in earlier sections may be difficult to combine in a coherent framework. The large number of possible combinations or configurations can make selecting an optimal method non-trivial, leading to delays or suboptimal design choices.

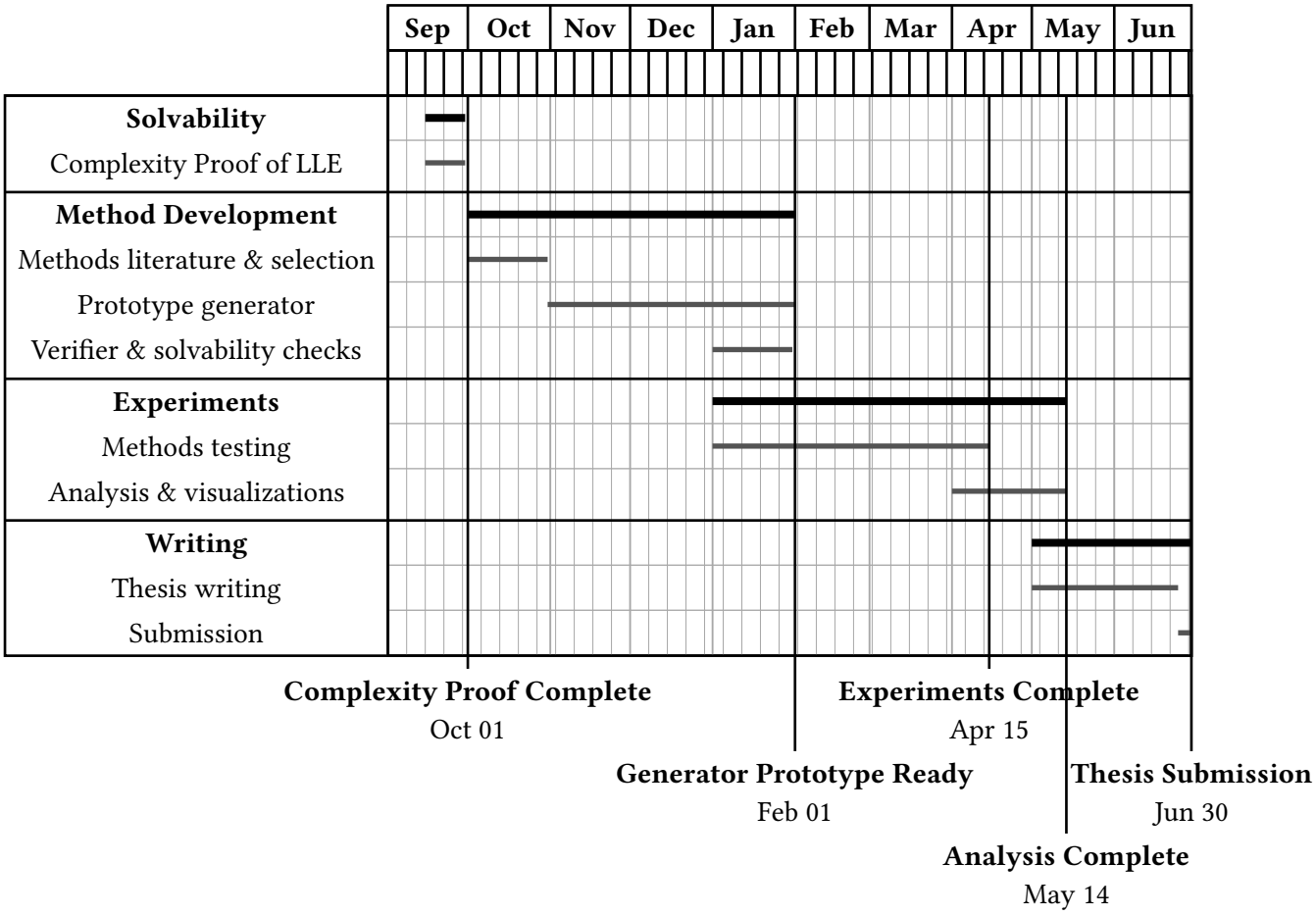
Third, the level generator itself may produce unsolvable levels, either due to inherent limitations in the generation process or insufficient constraints. Additional solvability-check mechanisms may be required, which could further increase implementation complexity.

Fourth, the implementation phase could be longer than planned. If significant development time is consumed by debugging or adapting methods, it may not be possible to conduct a full comparison between training on human-designed levels and training on procedurally generated ones.

Finally, there is a risk that curriculum learning may not improve agent performance in this setting, or that it may be inherently incompatible with the chosen PCG approach. In such a case, fallback strategies—such as simplifying the progression scheme or using alternative training paradigms—would need to be considered.

To mitigate these risks, contingency plans include: preparing alternative solvability-check approaches, narrowing down PCG method combinations early through pilot testing, integrating solvability constraints directly into the generation process, prioritizing core functionality in the implementation schedule, and developing secondary training strategies in case curriculum learning proves unsuitable.

4.6 Timeline


Sep
Oct
Nov
Dec
Jan
Feb
Mar
Apr
May
Jun
Solvability
Complexity Proof of LLE
Method Development
Methods literature & selection
Prototype generator
Verifier & solvability checks
Experiments
Methods testing
Analysis & visualizations
Writing
Thesis writing
Submission
Complexity Proof Complete
Oct 01
Experiments Complete
Apr 15
Generator Prototype Ready
Feb 01
Thesis Submission
Jun 30
Analysis Complete
May 14

Conclusion

This preparatory work has laid the foundation for a comprehensive investigation into procedural content generation for multi-agent reinforcement learning environments, with a specific focus on generating solvable levels that require meaningful cooperation between agents. Through our analysis of the Laser Learning Environment and related cooperative puzzle domains, we have identified key research questions and established a structured methodology to address the intersection of procedural content generation, multi-agent learning, and curriculum design.

5.1 Summary of Proposed Research

Our research plan addresses three fundamental challenges in generating content for cooperative MARL environments: ensuring solvability, promoting learnability, and enforcing cooperation requirements. The proposed work spans theoretical analysis, methodological development, and empirical validation.

From a theoretical perspective, we plan to establish the computational complexity landscape of multi-agent level generation. By extending the gadget-based reduction framework of Aloupis et al. to the LLE domain, we aim to demonstrate that determining the solvability of cooperative puzzle levels is NP-hard. This theoretical result will provide crucial context for understanding why generating solvable levels is computationally challenging and why heuristic or learning-based approaches are necessary in practice. The gadget constructions we propose to develop—including variable choice mechanisms, clause satisfaction patterns, and multi-agent coordination primitives—will offer a principled foundation for analyzing the complexity of other cooperative environments.

Methodologically, we have outlined a systematic evaluation of procedural content generation techniques across three major categories: noise-based methods, rule-based systems, and learning-based approaches. Our preliminary analysis suggests that while traditional PCG methods like cellular automata and noise functions can generate diverse spatial layouts, they will require significant augmentation to handle the semantic constraints and coordination dependencies inherent in MARL environments. Learning-based methods, particularly those employing reinforcement learning for content generation (PCGRL), appear promising for capturing the functional relationships between level elements and agent behaviors.

The proposed integration of curriculum learning principles into the level generation process represents a key methodological direction. By structuring the generation pipeline to produce levels of progressively increasing complexity—starting from simple coordination tasks and advancing to complex multi-agent puzzles—we aim to enable both the generator and the learning agents to develop capabilities incrementally. This approach mirrors successful curriculum learning strategies in other domains while addressing the unique challenges of cooperative behavior acquisition.

5.2 Expected Implications for Multi-Agent Learning

Our planned research has the potential for significant implications for the broader field of multi-agent reinforcement learning. The ability to generate diverse, solvable, and cooperation-requiring levels would address a critical bottleneck in MARL research: the scarcity of appropriate training environments. Hand-designed levels, while carefully crafted, are limited in number and may not capture the full spectrum of coordination challenges that agents need to master. Procedurally generated content offers a path toward more comprehensive evaluation and training of cooperative behaviors.

The emphasis on solvability verification will be particularly important for MARL applications. Unlike single-agent environments where unsolvable instances may simply result in failed episodes, multi-agent environments with unsolvable configurations can lead to pathological learning dynamics, where agents develop adversarial or non-cooperative behaviors in response to impossible coordination requirements. Our proposed multi-layered verification system—combining symbolic reasoning, agent-based testing, and hybrid validation—aims to provide robust guarantees that generated levels support meaningful learning experiences.

The cooperation requirements we plan to enforce through our generation framework directly address a fundamental challenge in MARL: ensuring that agents cannot succeed through purely individual strategies. By embedding structural dependencies that necessitate synchronized actions, role specialization, and mutual assistance, our generated levels would create authentic cooperative learning scenarios. This could be particularly valuable for benchmarking MARL algorithms and understanding their limitations in coordination-heavy domains.

5.3 Anticipated Challenges and Next Steps

The proposed research faces several significant challenges that will need to be addressed systematically. The theoretical complexity analysis, while conceptually grounded in established frameworks, will require careful adaptation to the specific mechanics and constraints of the LLE environment. Constructing appropriate gadgets that faithfully represent logical operations using multi-agent coordination primitives may prove more intricate than initially anticipated.

The development and evaluation of procedural generation methods presents additional complexity. Balancing the trade-offs between different PCG approaches—controllability versus diversity, computational efficiency versus output quality, structural coherence versus functional correctness—will require extensive experimentation and potentially novel hybrid architectures.

The integration of curriculum learning adds another layer of complexity, as it requires not only generating appropriate levels but also determining optimal sequencing and difficulty progression. The feedback loops between agent learning progress and level generation parameters will need careful tuning to avoid both trivial and impossibly difficult content.

Looking ahead, the immediate next steps involve implementing the theoretical complexity proof and establishing baseline PCG methods. This will be followed by the development of solvability verification systems and the integration of curriculum learning components. The experimental validation phase will require extensive computational resources and careful experimental design to ensure robust and reproducible results.

5.4 Concluding Remarks

This preparatory work establishes a roadmap for investigating the procedural generation of solvable levels in multi-agent reinforcement learning environments, representing a convergence of artificial intelligence, game design, computational complexity, and machine learning. By identifying the key theoretical questions and practical methodologies needed for this domain, we have laid the groundwork for contributions that could advance the development of more robust, generalizable, and thoroughly evaluated multi-agent systems.

The challenges we have outlined—balancing solvability with complexity, ensuring meaningful cooperation requirements, and scaling difficulty appropriately—are fundamental to many applications of multi-agent learning, from autonomous vehicle coordination to distributed robotics and beyond. As these systems become increasingly prevalent in real-world applications, the ability to generate diverse, challenging, and well-structured training environments becomes not just academically interesting, but practically essential.

The methodology we have proposed demonstrates the potential for procedural content generation, when properly constrained and guided by principled design choices, to serve as a powerful tool for advancing multi-agent learning research. By providing agents with rich, diverse, and progressively challenging cooperative scenarios, we aim to better understand the limits and capabilities of current MARL approaches while paving the way for more sophisticated coordination algorithms.

The intersection of procedural generation and multi-agent learning represents a fertile ground for research, with applications extending far beyond the specific domain of cooperative puzzle games. The theoretical insights, methodological frameworks, and practical tools we plan to develop will provide a foundation for continued exploration of this important and challenging research area. Success in this endeavor will contribute not only to the academic understanding of multi-agent coordination but also to the practical deployment of cooperative AI systems in real-world scenarios where reliable, adaptive, and robust coordination is essential.

Bibliography

[1] Y. Molinghen, R. Avalos, M. Van Achter, A. Nowé, and T. Lenaerts, “Laser Learning Environment: A New Environment for Coordination-Critical Multi-agent Tasks,” in *Artificial Intelligence and Machine Learning*, F. A. Oliehoek, M. Kok, and S. Verwer, Eds., Cham: Springer Nature Switzerland, 2025, pp. 135–154.

[2] Y. Molinghen, “LLE.” [Online]. Available: https://github.com/yamoling/lle

[3] J. von Neumann, *Theory of Self-Reproducing Automata*. Urbana, IL: University of Illinois Press, 1966.

[4] M. Gardner, “Mathematical Games: The fantastic combinations of John Conway's new solitaire game "life",” *Scientific American*, vol. 223, no. 4, pp. 120–123, 1970.

[5] M. Gumin, “Wave Function Collapse.” [Online]. Available: https://github.com/mxgmn/WaveFunctionCollapse

[6] R. Heese, “Quantum Wave Function Collapse for Procedural Content Generation,” *IEEE Computer Graphics and Applications*, vol. 44, no. 5, pp. 54–66, Sept. 2024, doi: 10.1109/mcg.2024.3447775.

[7] R. R. Torrado, A. Khalifa, M. C. Green, N. Justesen, S. Risi, and J. Togelius, “Bootstrapping Conditional GANs for Video Game Level Generation.” [Online]. Available: https://arxiv.org/abs/1910.01603

[8] A. Sarkar and S. Cooper, “Sequential Segment-based Level Generation and Blending using Variational Autoencoders.” [Online]. Available: https://arxiv.org/abs/2007.08746

[9] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, “PCGRL: Procedural Content Generation via Reinforcement Learning.” [Online]. Available: https://arxiv.org/abs/2001.09212

[10] G. Aloupis, E. D. Demaine, A. Guo, and G. Viglietta, “Classic Nintendo Games are (Computationally) Hard.” [Online]. Available: https://arxiv.org/abs/1203.1895

[11] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, “Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey.” [Online]. Available: https://arxiv.org/abs/2003.04960

[12] S. Risi and J. Togelius, “Increasing Generality in Machine Learning through Procedural Content Generation.” [Online]. Available: https://arxiv.org/abs/1911.13071

[13] A. Summerville and M. Mateas, “Super Mario as a String: Platformer Level Generation Via LSTMs.” [Online]. Available: https://arxiv.org/abs/1603.00930